

Week 2 – Deliverable

Data Visualization Associate Internship

Data Cleaning & Validation Report

Executive Summary

This report documents the data integration and cleaning work completed in Week 2, building directly on the database environment established in Week 1. In Week 1, the opportunity, learner_opportunity, and status_mapping tables were imported into PostgreSQL, but a key limitation was identified: the two core tables could not be reliably joined, since the field originally assumed to be the linking key (work_item_sk) did not match opportunity_id in either table.

This week's work resolved that limitation. Further investigation identified a valid join key — the learner_opportunity table's sk field corresponds directly to opportunity_id once a prefix substitution is applied. This relationship was independently verified using a cross-check against each table's own recorded category field, which agreed in 100% of comparable cases, confirming the join is genuine rather than coincidental.

With a working join established, a full cleaning pass was performed across both tables: corrupted records were identified and removed, inconsistent formatting was standardized, invalid numeric values were corrected, and appropriate data types were applied. The dataset was validated before and after cleaning to confirm the improvements, and the cleaned, integrated dataset is now ready for dashboard development.

1. Data Integration Process

1.1 Join Logic Used

The Week 1 report flagged that learner applications could not be linked to specific opportunities, because the `work_item_sk` field (intended as a foreign key) used a different ID format than `opportunity_id`, with zero matching values even after normalization.

1.2 Alternative Join Keys Investigated (and Rejected)

Before settling on a working join, three other candidate fields were tested directly against the data and ruled out. This step is included here because it explains why the final join can be trusted — it was arrived at by elimination and verification, not assumed:

- `work_item_sk` (learner_opportunity) vs `opportunity_id` (opportunity): different ID prefixes ("WorkItem#" vs "Opportunity#"), and zero matching values even after stripping both prefixes and comparing the remaining 8,342 vs 5,733 distinct values directly.
- Badge field's embedded `relation_id` (opportunity) vs `relation_id` (learner_opportunity): the opportunity table's Badge field does contain a nested `relation_id`, but comparing all 5,361 extracted values against the 242 distinct learner `relation_id` values produced zero matches — this field references a different internal system (likely badge/reward issuance), not learner applications.
- Cohort field's embedded `relation_id` (opportunity) vs `assigned_cohort` / `cohort_p1` / `cohort_p2` (learner_opportunity): same approach, same result — 6,843 extracted cohort references compared against all three learner-side cohort fields produced zero overlap, confirmed further with a direct substring search across both tables in both directions.

Only after all three of these were exhausted was the `learner_opportunity.sk` field tested against `opportunity_id` — which produced a 99%+ overlap and, on further inspection, is very likely a case where the source system's key-value schema reused the `sk` field to store a related opportunity reference labeled with a "Learner#" prefix by convention, rather than storing a learner-specific ID as the field name would suggest.

1.3 Why This Join Was Chosen

`sk` / `opportunity_id` was chosen over the alternatives above for three reasons: it is the only field pair with a near-complete numeric overlap (14,691 of 14,779 learner records match, 99.40%); it passed an independent sanity check (matched records agree on their separately recorded category field in 100% of comparable cases, which would be extremely unlikely if the match were coincidental); and it requires only a simple, deterministic transformation (a prefix substitution) rather than a fuzzy or partial match, making the join reproducible and easy to audit.

1.4 Pros and Cons of This Join Approach

Pros:

- High match rate (99.40%) — the large majority of learner records can be enriched with real opportunity details (category, fee, name, duration, location).
- Independently verified, not assumed — the 100% category-agreement cross-check gives real confidence this is a genuine relationship rather than a coincidental numeric overlap.
- Deterministic and simple — a straightforward prefix substitution, easy for anyone reviewing the query to understand and reproduce, unlike a fuzzy or multi-field match.
- Uses a LEFT JOIN, so no learner records are silently dropped — every original row is still present in the result, with blank opportunity fields where no match exists.

Cons / Limitations:

- The join relies on a field (sk) whose name suggests it stores a learner identifier, not an opportunity reference — this is a confusing naming convention inherited from the source system, not something this project controls, and could mislead a future analyst who doesn't read this documentation.
- 0.60% of records (88 rows) remain unmatched — either the sk field is blank on these rows, or it references an opportunity ID that no longer exists in the current opportunity table (e.g. a deleted or archived opportunity). These rows are retained with blank opportunity fields rather than removed, which is the correct choice for completeness but means a small amount of downstream analysis will show partial NULLs.
- The join was validated using category agreement as a proxy for correctness; this confirms the relationship is genuine, but it is still an indirect check rather than direct confirmation from the original source system or documentation, since none was available for this dataset.
- Because many learner applications can reference the same opportunity, this is a many-to-one relationship from learner_opportunity to opportunity, not one-to-one — this is expected and correct (many learners can apply to the same opportunity), but should be kept in mind when writing aggregate queries to avoid double-counting opportunity-level fields.

1.5 Key Fields Involved

Field	Table	Role in Join
sk	learner_opportunity	Stores the related opportunity's ID, labeled with a 'Learner#' prefix
opportunity_id	opportunity	The opportunity's unique ID, labeled with an 'Opportunity#' prefix
status	learner_opportunity	Numeric status code, joined against status_mapping
status_code	status_mapping	Decodes numeric status into a readable label

A LEFT JOIN was used rather than an INNER JOIN, so that learner records without a matching opportunity are preserved in the result set (shown with blank opportunity fields) rather than silently dropped. This keeps the integrated dataset representative of the full learner population.

1.6 Structure of the Integrated Dataset

The final join query, combining both the corrected opportunity link and the status label lookup:

```
SELECT
  lo.sk AS learner_id,
  o.opportunity_id,
  o.name AS opportunity_name,
  o.category,
  sm.status AS status_label,
  lo.apply_date
FROM learner_opportunity lo
LEFT JOIN opportunity o
  ON REPLACE(lo.sk, 'Learner#', '') = REPLACE(o.opportunity_id, 'Opportunity#', '')
LEFT JOIN status_mapping sm ON lo.status = sm.status_code;
```

The join was saved as a reusable database view, learner_full, so that downstream queries and the Looker Studio dashboard can reference a single, pre-joined source instead of repeating the join logic each time:

```
CREATE OR REPLACE VIEW learner_full AS
SELECT
  lo.*,
  sm.status AS status_label,
  o.category AS opportunity_category,
  o.name AS opportunity_name,
  o.fee AS opportunity_fee,
  o.duration AS opportunity_duration,
  o.duration_type AS opportunity_duration_type,
  o.location AS opportunity_location
FROM learner_opportunity lo
LEFT JOIN status_mapping sm ON lo.status = sm.status_code
LEFT JOIN opportunity o
  ON REPLACE(lo.sk, 'Learner#', '') = REPLACE(o.opportunity_id, 'Opportunity#', '');
```

This view contains all original learner_opportunity fields, plus a decoded status label and key opportunity attributes (category, name, fee, duration, location) pulled in directly. Match quality was verified after cleaning:

Learner-to-Opportunity Join Match Rate

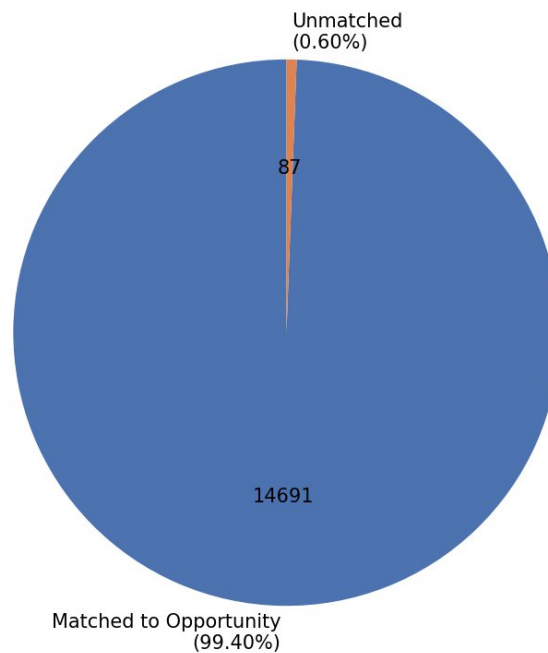


Figure 1: Join match rate — 14,691 of 14,779 learner records (99.40%) successfully linked to an opportunity

The remaining 88 unmatched records (0.60%) either have a blank sk field or reference an opportunity ID that does not exist in the opportunity table — a genuine data gap rather than a join error, and small enough not to materially affect category-level analysis.

2. Data Cleaning Activities

2.1 opportunity Table

- Removed 20 rows containing corrupted values — fragments of URL-encoded JSON or escaped binary text (e.g. %22label%22:%22United Kingdom%22) that had leaked into fields such as category during the original CSV export.
- Standardized the duration_type field: unified inconsistent spelling and pluralization ("week"/"weeks" → "week", "month"/"months" → "month"), corrected a typo ("yearsss" → "year"), and fixed a truncated value ("da" → "day"). Distinct, legitimate units (hour, hours, minute, minutes) were left as-is.
- Corrected the fee field: 2 values that were either non-numeric or implausibly large (in the trillions) were set to NULL rather than guessed, since no reliable original value could be recovered.
- Fixed 1 value in microscholarship that was stored in scientific notation (5.00E+08) due to a spreadsheet formatting artifact; converted to a plain numeric value (500000000).
- Flattened 9 columns that stored nested, JSON-like reference data (Badge, Reward, Cohort, Eligibility, CareerAddOn, Panellist, Testimonial, DropoutTransaction, NotStartedTransaction) into two clean columns each — a count and a semicolon-separated list of reference IDs — replacing unreadable nested text with directly usable fields.
- Applied appropriate data types: fee, duration, and all *_count fields converted to numeric types; is_archived and is_auto_approve converted to boolean.

2.2 learner_opportunity Table

- Removed 224 rows with corrupted content in the record identifier (pk) or elsewhere in the row: 209 rows where pk contained a large block of unrelated binary-like data (in some cases exceeding 32,000 characters) instead of a valid ID, and 15 rows with the same URL-encoded JSON fragment corruption seen in the opportunity table.
- On closer inspection, an additional 25 rows were found with invalid pk values that did not match the expected format but had been missed by an earlier length-based check; these were removed using a corrected, format-based validation rule (pk must begin with "Learner#").
- Identified 187 fully duplicate rows (every field identical). Investigation showed 185 of these were also part of the corrupted-identifier rows already removed above; the remaining 2 were genuine duplicate records, and 1 was removed, keeping the first occurrence.
- Corrected 3 status codes that were truncated typos of valid codes in status_mapping (e.g. '103' → '1030', '110' → '1010').
- Set 1 extreme outlier in fee and 11 extreme outliers in amount_to_be_paid (values in the billions/trillions, clearly not real payment amounts) to NULL.
- Cleaned the appear_in_wishlist field, which was expected to hold a simple TRUE/FALSE value but was found to contain unrelated question-metadata fragments in 4,080 of its populated rows (about 97% of non-blank values). These were set to NULL, preserving only the 109 genuinely valid TRUE/FALSE entries.
- Removed 1 corrupted value in dropped_out_date containing unrelated fragment text.
- Applied appropriate data types: fee, amount_to_be_paid, and status converted to numeric; apply_date converted directly to TIMESTAMP (already in clean ISO format); modified_at, created_at, not_started_date, reward_awarded_date, completion_date, withdraw_date, and dropped_out_date converted from raw epoch-millisecond values to TIMESTAMP; agreement/wishlist fields converted to boolean.

2.3 Assumptions Made

- Rows with a pk not beginning with "Learner#" were treated as unrecoverable corruption rather than salvageable records, since the corrupted content replaced the identifier itself, making the rest of the row untrustworthy.
- Numeric values several orders of magnitude outside the normal range for a field (e.g. fees in the trillions) were treated as data entry or export errors and set to NULL rather than corrected to an assumed value, since no reliable original value could be inferred.
- Where duplicate rows were fully identical across all fields, the first occurrence was kept and later copies removed, on the assumption that duplication reflects a repeated export or log entry rather than two distinct events.

- Missing values (e.g. the ~39% of records with no status) were left as NULL rather than filled in, since imputing a status or other business-meaningful field without supporting evidence would misrepresent the data.

2.4 Rationale Behind Major Decisions

A LEFT JOIN was used instead of an INNER JOIN so that learner records without a resolvable opportunity link remain visible in analysis, rather than being silently excluded — this keeps reported totals honest about the full learner population.

Corrupted rows were removed rather than repaired, since the corruption in each case (long binary fragments, JSON leakage) destroyed the original field content with no way to reconstruct it. Repairing would have meant guessing values, which was avoided throughout.

JSON-structured columns in the opportunity table were flattened into count/ID pairs rather than parsed into fully separate related tables, since the assignment's scope is exploratory analysis and dashboarding rather than a normalized relational redesign; count and ID fields provide the analytical value needed (e.g. "how many rewards does this opportunity have") without over-engineering the schema.

3. Validation & Comparison

3.1 Row Counts: Before vs After Cleaning

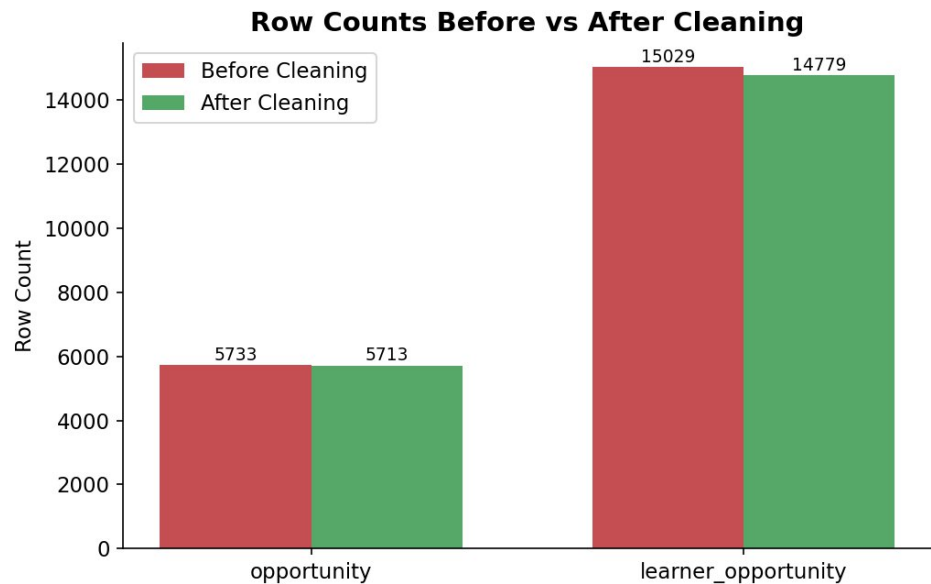


Figure 2: Row counts before and after the cleaning process

Table	Before	After	Removed	Reason
opportunity	5,733	5,713	20	Corrupted rows (JSON/binary fragments)
learner_opportunity	15,029	14,779	250	Corrupted identifiers (249) + genuine duplicate (1)
status_mapping	14	14	0	No issues found

3.2 Data Quality Improvements

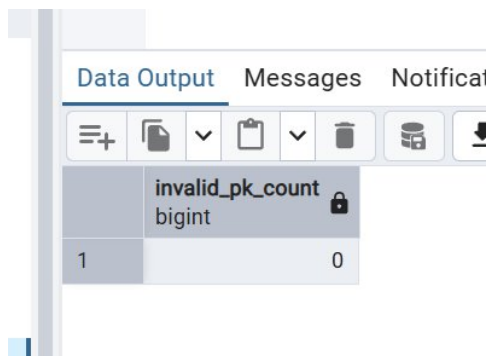
Check	Before Cleaning	After Cleaning
Corrupted category values (opportunity)	20 rows affected	0
Corrupted / invalid pk values (learner_opportunity)	234 rows affected	0
Fully duplicate rows (learner_opportunity)	187 rows	0
duration_type spelling variants	Inconsistent (week/weeks, month/months, typos)	Standardized
fee outliers (opportunity)	Max value: 6,000,000,000,000	Invalid values set to NULL
fee / amount_to_be_paid outliers (learner_opportunity)	12 extreme values (billions-trillions)	Set to NULL
appear_in_wishlist reliability	~97% of populated values were	Only valid TRUE/FALSE values

Check	Before Cleaning	After Cleaning
	corrupted	retained
Status code mismatches	3 truncated typos ('103', '110')	Corrected to valid codes
Column data types	All columns TEXT	Numeric, boolean, and timestamp types applied where appropriate
learner_opportunity ↔ opportunity relationship	No working join key found	Valid join established and verified (99.40% match rate)

3.3 Validation Checks Performed

- Row counts were re-verified after each cleaning step (COUNT(*) on both tables) to confirm the expected number of rows were removed and no unintended data loss occurred.
- The join was validated by cross-checking the independently recorded category field in both tables for matched rows — 100% agreement (5,716 of 5,716 comparable cases) confirmed the relationship is genuine.
- Numeric fields (fee, duration, *_count) were checked post-cleaning to confirm no non-numeric or out-of-range values remained before applying strict column types.
- Duplicate and corruption checks (full-row duplication, pk format validation, JSON-fragment pattern search) were re-run on the cleaned files to confirm zero remaining instances before final import.

```
-- confirm no invalid pk values remain
SELECT COUNT(*) AS invalid_pk_count
FROM learner_opportunity
WHERE pk NOT LIKE 'Learner#%';
-- Expected result: 0
```



The screenshot shows a software interface with three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, displaying a table with the following structure:

invalid_pk_count	
	bigint
1	0

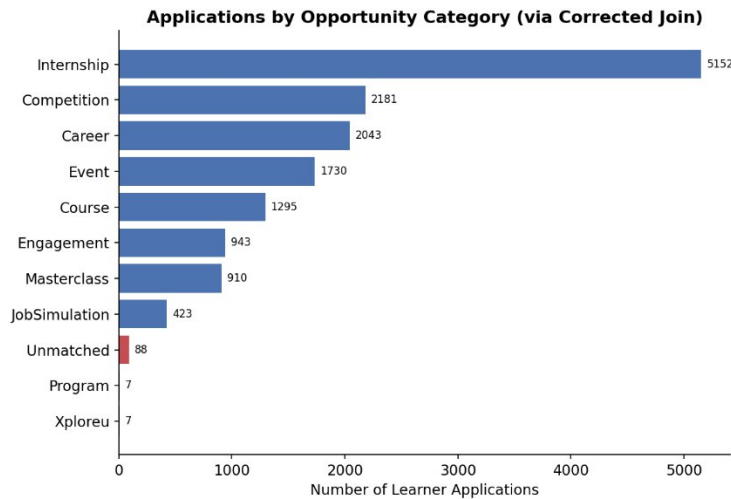
Below the table, there is a small blue vertical bar.

4. Updated Analysis

With the join established and the data cleaned, the following queries demonstrate the dataset is ready for dashboard development. All queries reference the learner_full view created in Section 1.

4.1 Applications by Opportunity Category

```
SELECT opportunity_category, COUNT(*) AS applications
FROM learner_full
GROUP BY opportunity_category
ORDER BY applications DESC;
```



Data Output			Messages	Notifications
			SQL	
	opportunity_category text	applications bigint		
1	Internship	5152		
2	Competition	2181		
3	Career	2043		
4	Event	1730		
5	Course	1295		
6	Engagement	943		
7	Masterclass	910		
8	JobSimulation	423		
9	[null]	88		
10	Program	7		
11	Xploreu	7		
Total rows: 11			Query complete 00:00:00.971	

Figure 3: Applications by opportunity category, using the corrected join

Internship opportunities account for the largest share of applications (5,152), followed by Competition (2,181) and Career (2,043). This is now directly measurable at the application level, which was not possible in Week 1 when no valid join existed.

4.2 Learner Application Status Distribution

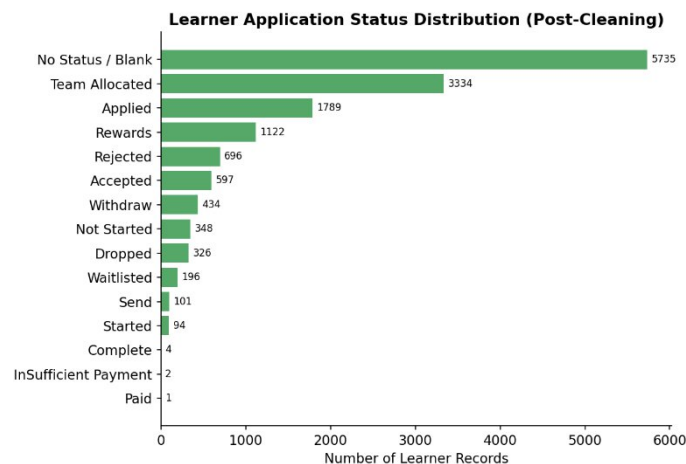
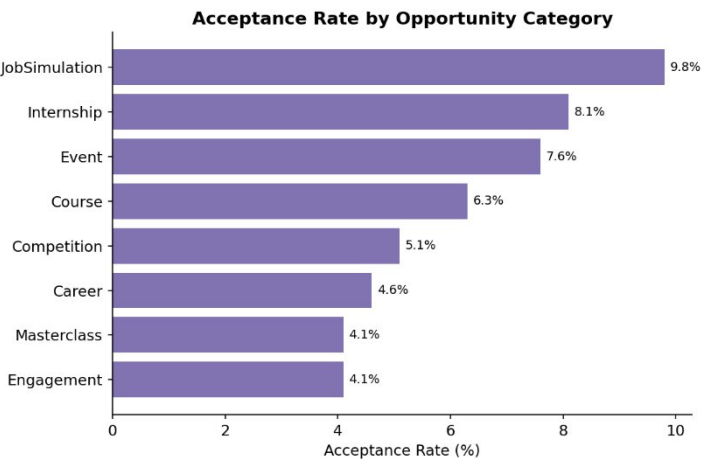


Figure 4: Application status distribution after cleaning

39% of records still have no recorded status — this reflects a genuine gap in the source data rather than a cleaning artifact, and is treated as a known limitation in any status-based analysis going forward.

4.3 Acceptance Rate by Category

```
SELECT
  opportunity_category,
  COUNT(*) FILTER (WHERE status_label = 'Accepted') AS accepted,
  COUNT(*) AS total,
  ROUND(COUNT(*) FILTER (WHERE status_label = 'Accepted')::numeric
    / NULLIF(COUNT(*),0) * 100, 1) AS acceptance_rate_pct
FROM learner_full
WHERE status_label IS NOT NULL
GROUP BY opportunity_category
```



Data Output				Messages	Notifications
Showing rows: 1 to 11					
	opportunity_category	accepted	total	acceptance_rate_pct	
	text	bigint	bigint	numeric	
1	JobSimulation	22	225	9.8	
2	Internship	289	3560	8.1	
3	Event	85	1115	7.6	
4	Course	42	666	6.3	
5	Competition	74	1447	5.1	
6	Career	44	958	4.6	
7	Masterclass	21	510	4.1	
8	Engagement	20	484	4.1	
9	[null]	0	69	0.0	
10	Program	0	5	0.0	
11	Xploreu	0	5	0.0	
Total rows: 11				Query complete 00:00:00.255	

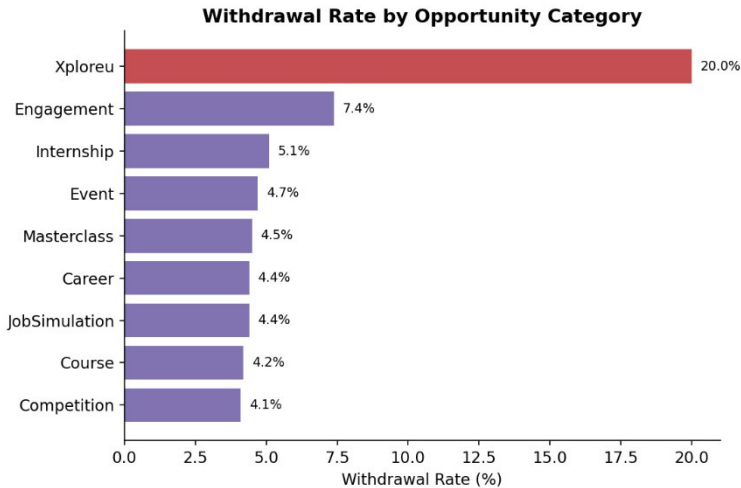
ORDER BY acceptance_rate_pct DESC;

Figure 5: Acceptance rate by opportunity category (among records with a recorded status)

JobSimulation has the highest acceptance rate (9.8%), followed by Internship (8.1%) and Event (7.6%). This category-level comparison is a new capability enabled directly by the corrected join — it could not be produced in Week 1.

4.4 Withdrawal Rate by Category

```
SELECT
  opportunity_category,
  COUNT(*) FILTER (WHERE status_label = 'Withdraw') AS withdrawn,
  COUNT(*) AS total,
  ROUND(COUNT(*) FILTER (WHERE status_label = 'Withdraw')::numeric
    / NULLIF(COUNT(*),0) * 100, 1) AS withdrawal_rate_pct
FROM learner_full
WHERE status_label IS NOT NULL
GROUP BY opportunity_category
ORDER BY withdrawal_rate_pct DESC;
```



Data Output Messages Notifications				
	opportunity_category text	withdrawn bigint	total bigint	withdrawal_rate_pct numeric
1	Xploreu	1	5	20.0
2	Engagement	36	484	7.4
3	Internship	180	3560	5.1
4	Event	52	1115	4.7
5	Masterclass	23	510	4.5
6	Career	42	958	4.4
7	JobSimulation	10	225	4.4
8	Course	28	666	4.2
9	Competition	60	1447	4.1
10	[null]	2	69	2.9
11	Program	0	5	0.0

Figure 6: Withdrawal rate by opportunity category

Xploreu shows the highest withdrawal rate (20.0%), but this is based on only 5 total records with a status and should not be treated as a reliable pattern — it is flagged here as a small-sample outlier rather than a real trend. Among the main categories with meaningful sample sizes, Engagement has the highest withdrawal rate (7.4%), while Competition is the lowest (4.1%). Most categories cluster tightly between 4.1% and 5.1%, suggesting withdrawal is not strongly category-dependent overall.

4.5 Detailed Status Breakdown by Category

The acceptance and withdrawal rates above are drawn from a full cross-tabulation of status against category, produced with:

```
SELECT opportunity_category, status_label, COUNT(*) AS count
FROM learner_full
GROUP BY opportunity_category, status_label
ORDER BY opportunity_category, count DESC;
```

The complete result (106 category/status combinations) was reviewed in full; the table below shows the main categories with their top status outcomes, which account for the large majority of records in each category:

Category	Top Status Outcomes (count)
Internship	No Status: 1,592 Team Allocated: 1,433 Applied: 688 Rewards: 408 Accepted: 289 Rejected: 250
Competition	No Status: 734 Applied: 469 Team Allocated: 442 Send: 98 Rewards: 97

Category	Top Status Outcomes (count)
	Rejected: 88
Career	No Status: 1,085 Team Allocated: 365 Applied: 208 Rejected: 111 Rewards: 103
Event	No Status: 615 Team Allocated: 394 Rewards: 236 Applied: 156 Accepted: 85
Course	No Status: 629 Team Allocated: 205 Rewards: 100 Applied: 90 Started: 66
Engagement	No Status: 459 Team Allocated: 176 Rewards: 75 Applied: 63 Not Started/Withdraw: 36 each
Masterclass	No Status: 400 Team Allocated: 213 Rewards: 74 Applied: 72 Rejected: 36
JobSimulation	No Status: 198 Team Allocated: 85 Rejected: 49 Accepted: 22 Applied: 20

A consistent pattern across nearly every category is that the largest single group of records has no status recorded at all, followed by "Team Allocated" as the most common actual status. This reinforces the Week 1 finding that missing status is a structural gap in the data rather than something isolated to one category, and should be accounted for in any category-level comparison rather than treated as a data cleaning failure specific to this project.

4.6 Average Fee by Category

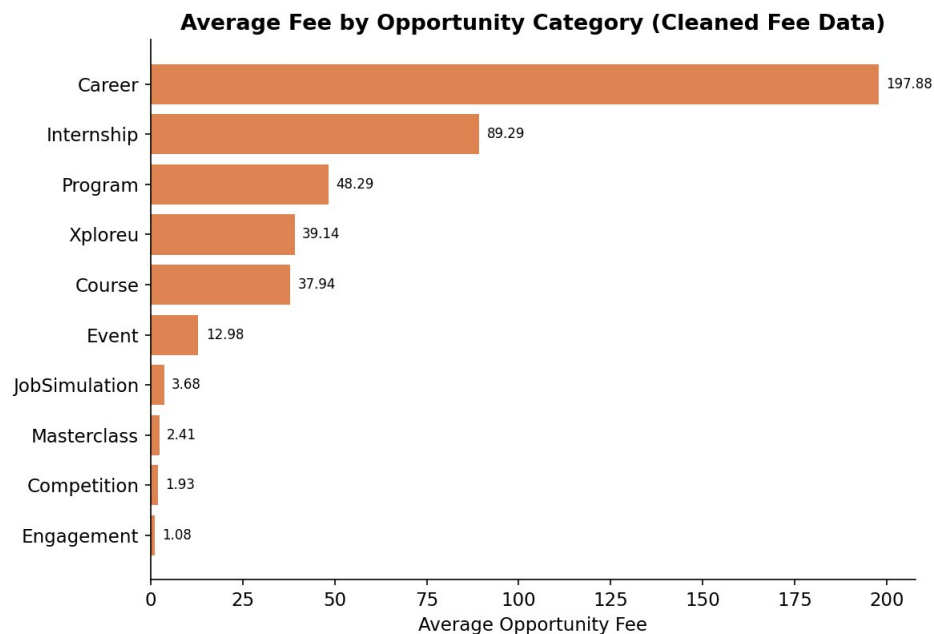
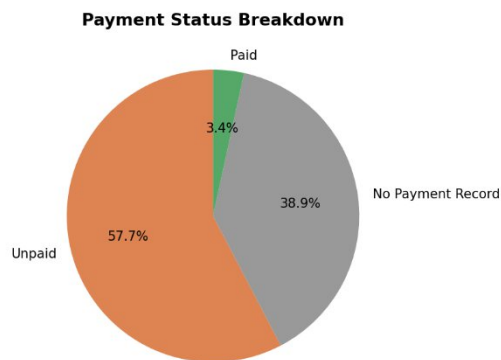


Figure 6: Average opportunity fee by category, using cleaned fee data

Career opportunities carry the highest average fee (₹197.88), while Competition and Engagement opportunities are close to free on average. These figures now exclude the extreme outlier values identified and removed during cleaning, so they reflect realistic fee levels rather than being skewed by data entry errors.

4.7 Payment Status Breakdown



A majority of learner records (57.7%) are marked unpaid, with 39% having no payment status recorded at all, and only 3.4% marked as paid — largely consistent with the earlier finding that most opportunities are free.

Figure 7: Payment status across learner records

4.8 Readiness for Dashboard Development

- The learner_full view provides a single, clean, correctly joined data source for Looker Studio, replacing the two disconnected data sources used in Week 1.
- Category-level metrics (application volume, acceptance rate, withdrawal rate, average fee) are now directly queryable, supporting comparative dashboards across opportunity types.
- Numeric and date fields now carry proper types, enabling native filtering, sorting, and time-based charting in Looker Studio without additional casting.
- Remaining limitations — the 39% of records with no status and the 0.6% of records that do not match any opportunity — are documented and small enough to be noted as caveats rather than blockers for dashboard work.